

Banker's Algorithm

Deadlock Avoidance in Operating Systems

Motivation

Problem

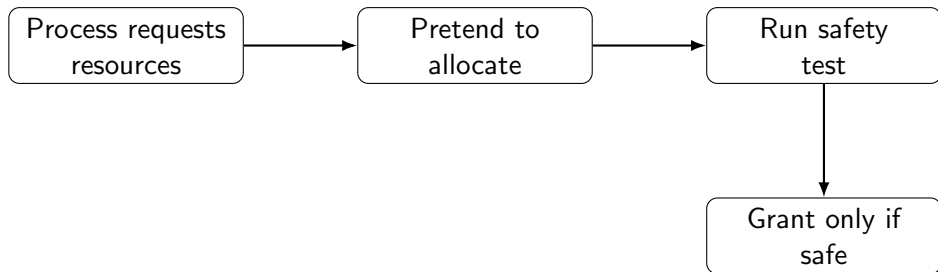
In a system with multiple processes and reusable resources, granting a resource request too early can lead to deadlock.

Goal

Decide whether a request is safe before committing it.

- Resources have multiple instances, such as printers, memory blocks, or devices.
- Processes declare their maximum possible demand in advance.
- The operating system grants a request only if the system can still finish all processes.

Core Idea



Banker's Principle

A resource request is granted only if there exists at least one order in which every process can complete.

System Model

Assume:

- There are n processes: $P_0, P_1, \dots, P_{n-1}$.
- There are m resource types: $R_0, R_1, \dots, R_{m-1}$.
- Each resource type may have multiple identical instances.
- Each process declares its maximum resource demand before execution.

Deadlock Avoidance

The algorithm does not wait for deadlock to happen. It avoids entering states from which deadlock could become unavoidable.

When Does Deadlock Occur?

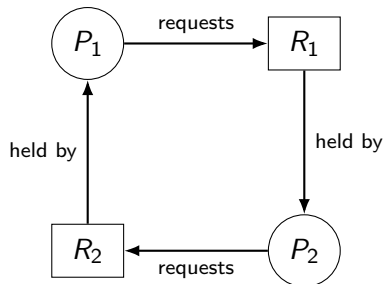
A deadlock can occur only when all four Coffman conditions hold at the same time:

- ① **Mutual exclusion**: at least one resource cannot be shared.
- ② **Hold and wait**: a process holds resources while waiting for more.
- ③ **No preemption**: resources cannot be forcibly taken away.
- ④ **Circular wait**: processes form a cycle, each waiting for a resource held by the next.

Important

If any one of these conditions is broken, deadlock cannot occur.

Deadlock as a Waiting Cycle



Cycle Interpretation

P_1 is waiting for R_1 , but R_1 is held by P_2 . At the same time, P_2 is waiting for R_2 , but R_2 is held by P_1 .

Structure	Meaning
$Available[m]$	Number of currently available instances of each resource type.
$Max[n][m]$	Maximum demand of each process.
$Allocation[n][m]$	Resources currently allocated to each process.
$Need[n][m]$	Remaining resources each process may still request.
$Request_i[m]$	Current request made by process P_i .

$$Need[i][j] = Max[i][j] - Allocation[i][j]$$

Safe State

A system state is safe if there exists a sequence of all processes

$$\langle P_a, P_b, \dots, P_z \rangle$$

such that each process can obtain its remaining needs, finish, and release its resources for the next process.

- Safe means deadlock can be avoided.
- Unsafe does not mean deadlock has already occurred.
- Unsafe means the system no longer has a guaranteed completion order.

Safety Algorithm

- 1 Set $Work = Available$.
- 2 Set $Finish[i] = false$ for every process P_i .
- 3 Find an unfinished process P_i such that

$$Need_i \leq Work$$

- 4 If found, pretend P_i completes:

$$Work = Work + Allocation_i$$

and set $Finish[i] = true$. Repeat step 3.

- 5 If every $Finish[i]$ is true, the system is safe.

Resource-Request Algorithm

For a request $Request_i$ from process P_i :

- 1 Check that the process is not asking for more than it declared:

$$Request_i \leq Need_i$$

- 2 Check that the resources are currently available:

$$Request_i \leq Available$$

- 3 Pretend to allocate:

$$Available = Available - Request_i$$

$$Allocation_i = Allocation_i + Request_i$$

$$Need_i = Need_i - Request_i$$

- 4 Run the safety algorithm. Grant only if the resulting state is safe.

Example: Current State

Resource types: A , B , and C

$$Available = \langle 3, 3, 2 \rangle$$

	A	B	C
P_0	0	1	0
P_1	2	0	0
P_2	3	0	2
P_3	2	1	1
P_4	0	0	2

Table: Allocation

	A	B	C
P_0	7	5	3
P_1	3	2	2
P_2	9	0	2
P_3	2	2	2
P_4	4	3	3

Table: Max

Example: Need Matrix

$$\text{Need} = \text{Max} - \text{Allocation}$$

	A	B	C
P_0	7	4	3
P_1	1	2	2
P_2	6	0	0
P_3	0	1	1
P_4	4	3	1

Initial Work

$$\text{Work} = \text{Available} = \langle 3, 3, 2 \rangle$$

Example: Safety Check

Step	Process	Need	Work before	Work after	Reason
1	P_1	$\langle 1, 2, 2 \rangle$	$\langle 3, 3, 2 \rangle$	$\langle 5, 3, 2 \rangle$	$Need_1 \leq Work$
2	P_3	$\langle 0, 1, 1 \rangle$	$\langle 5, 3, 2 \rangle$	$\langle 7, 4, 3 \rangle$	$Need_3 \leq Work$
3	P_4	$\langle 4, 3, 1 \rangle$	$\langle 7, 4, 3 \rangle$	$\langle 7, 4, 5 \rangle$	$Need_4 \leq Work$
4	P_0	$\langle 7, 4, 3 \rangle$	$\langle 7, 4, 5 \rangle$	$\langle 7, 5, 5 \rangle$	$Need_0 \leq Work$
5	P_2	$\langle 6, 0, 0 \rangle$	$\langle 7, 5, 5 \rangle$	$\langle 10, 5, 7 \rangle$	$Need_2 \leq Work$

Safe Sequence

$\langle P_1, P_3, P_4, P_0, P_2 \rangle$

Example: Handling a Request

Suppose P_1 requests

$$Request_1 = \langle 1, 0, 2 \rangle$$

Check Limits

$$\langle 1, 0, 2 \rangle \leq Need_1 = \langle 1, 2, 2 \rangle$$

$$\langle 1, 0, 2 \rangle \leq Available = \langle 3, 3, 2 \rangle$$

Pretend Allocation

$$Available = \langle 2, 3, 0 \rangle$$

$$Allocation_1 = \langle 3, 0, 2 \rangle$$

$$Need_1 = \langle 0, 2, 0 \rangle$$

Then run the safety algorithm on the new state.

Request Result

After pretending to grant $Request_1 = \langle 1, 0, 2 \rangle$:

Process	Need	Work before	Work after
P_1	$\langle 0, 2, 0 \rangle$	$\langle 2, 3, 0 \rangle$	$\langle 5, 3, 2 \rangle$
P_3	$\langle 0, 1, 1 \rangle$	$\langle 5, 3, 2 \rangle$	$\langle 7, 4, 3 \rangle$
P_4	$\langle 4, 3, 1 \rangle$	$\langle 7, 4, 3 \rangle$	$\langle 7, 4, 5 \rangle$
P_0	$\langle 7, 4, 3 \rangle$	$\langle 7, 4, 5 \rangle$	$\langle 7, 5, 5 \rangle$
P_2	$\langle 6, 0, 0 \rangle$	$\langle 7, 5, 5 \rangle$	$\langle 10, 5, 7 \rangle$

Decision

The request can be granted because the resulting state is safe.

Unsafe State vs. Deadlock

Unsafe State

There is no guaranteed safe sequence from the current state.

Deadlock is possible, but not necessarily present.

Deadlock

A set of processes are permanently waiting for resources held by one another.

No process in the set can proceed.

Key Distinction

Banker's algorithm prevents unsafe allocations, so it avoids deadlock before deadlock forms.

Complexity

Let:

- n be the number of processes.
- m be the number of resource types.

Safety Algorithm

In the worst case, the algorithm scans up to n processes across up to n successful completions, comparing m resource types each time.

$$O(n^2m)$$

Resource-Request Algorithm

It performs constant vector updates plus one safety check, so its dominant cost is also $O(n^2m)$.

Advantages and Limitations

Advantages

- Avoids deadlock before it occurs.
- Supports multiple resource instances.
- Gives a clear correctness condition: safety.

Limitations

- Requires maximum demand in advance.
- Can delay requests even when resources are free.
- Adds runtime overhead to each request.

Common Exam Pitfalls

- Confusing *Max*, *Allocation*, and *Need*.
- Forgetting that $Need = Max - Allocation$.
- Granting a request only because $Request_i \leq Available$.
- Treating an unsafe state as the same thing as deadlock.
- Releasing a process's *Need* instead of its *Allocation* when it finishes.

Rule of Thumb

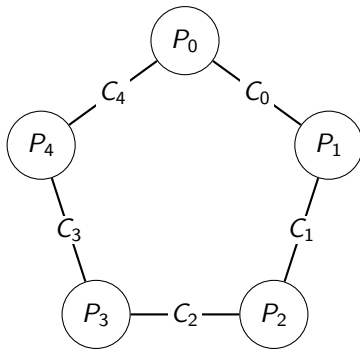
When a process finishes, it releases what it already holds:

$$Work = Work + Allocation_i$$

Classic Problem: Dining Philosophers

Setup

Five philosophers sit around a circular table. Between every pair of philosophers is one chopstick. A philosopher needs both adjacent chopsticks to eat.



Dining Philosophers: Why Deadlock Happens

A simple but dangerous strategy:

- 1 Pick up the left chopstick.
- 2 Pick up the right chopstick.
- 3 Eat.
- 4 Put both chopsticks down.

Deadlock Scenario

If all philosophers pick up their left chopstick at the same time, every philosopher holds one chopstick and waits forever for the right chopstick.

- Mutual exclusion: each chopstick can be held by only one philosopher.
- Hold and wait: each philosopher holds one chopstick while waiting.
- No preemption: chopsticks are not forcibly taken away.
- Circular wait: each philosopher waits for the next philosopher's chopstick.

Semaphore Solution

Use one binary semaphore per chopstick, plus $room = 4$ to limit how many philosophers may compete at once.

```
semaphore room = 4;  
semaphore chopstick[5] = {1, 1, 1, 1, 1};
```

```
philosopher i:  
    while true:  
        think()  
        wait(room)  
        wait(chopstick[i])  
        wait(chopstick[(i + 1) mod 5])  
        eat()  
        signal(chopstick[(i + 1) mod 5])  
        signal(chopstick[i])  
        signal(room)
```

Why the Semaphore Solution Works

- The chopstick semaphores preserve mutual exclusion.
- The *room* semaphore prevents all five philosophers from holding one chopstick at once.
- This breaks circular wait, so the deadlock cycle cannot form.
- Philosophers may still wait, but the system always has a path forward.

Connection to Banker's Algorithm

Banker's algorithm avoids unsafe resource allocations in a general resource system. The Dining Philosophers solution avoids one specific unsafe pattern by controlling entry into the resource competition.

Summary

- Banker's algorithm is a deadlock-avoidance algorithm.
- It uses declared maximum needs to test whether states are safe.
- A safe state has at least one process completion sequence.
- A request is granted only after pretending to allocate and confirming safety.
- The main cost is the safety check: $O(n^2m)$.

Questions?